

Lexique pour le fichier de configuration OpenADOM

Inventaire des mots clefs du fichier de configuration OpenADOM

VARLOTEAUX Lucile TCHERNIATINSKY Philippe

2022-10-10

Table des matières

1	Lexique	3
1.1	OpenADOM	3
1.2	L'application	3
1.3	Les étiquettes (<i>tags</i>)	3
1.4	Les référentiels (<i>references</i>)	4
1.4.1	Les colonnes du fichier	5
1.4.2	Les colonnes calculées	5
1.4.3	Les colonnes dynamiques	5
1.5	Définition de référentiels hiérarchiques	6
1.6	Les types de données (<i>dataTypes</i>)	6
1.6.1	data	7
1.6.1.1	Components	7
1.6.1.2	ComputedComponents	8
1.6.2	format	8
1.6.3	Authorization	9
1.6.4	Repository	10
1.7	Sections génériques	11
1.7.1	InternationalisationName	11
1.7.2	Groovy	11
1.7.3	Checker	12
1.7.3.1	Contenu de la section params :	12
1.7.3.2	Computation	13
1.7.3.3	Presence constraint	13

1 Lexique

Vous trouverez ici une liste des termes immuables, se trouvant dans le *fichier de configuration*, accompagnés d'une définition et d'exemple.

1.1 OpenADOM

version : nous l'utilisons pour la version de OpenADOM (actuellement, c'est la version 1).

1.2 L'application

application : dans cette partie nous décrirons l'application.

- **version** : Nous changeons la version de l'application avant d'importer les *fichiers de configuration* sur OpenAdom.
- **defaultLanguage** : On peut définir ici une langue d'affichage par défaut en utilisant les [abréviations](#) du nom de la langue. (ex : français -> fr, anglais -> en)
- **name** : Le nom de l'application qui sera enregistré en base de données.
- **internationalizationName** : L'affichage du nom (**name**) en fonction de la langue : (cf.internationalisationName)

1.3 Les étiquettes (*tags*)

tags: Création d'un regroupements sous une étiquette permettant de filtré l'affichages des listes des **references** et des **datatypes**. Mais aussi les colonnes, les colonnes calculées, les colones dynamiques d'une **reference** et les data, les components et les computedComponents d'un **datatype**.

L'étiquette `__hidden__` est une étiquette qui n'a pas besoin d'être mise dans la liste de création. Nous l'utiliserons pour les données que l'on veut enregistrer en base mais que l'on ne veut pas rendre accessible à l'utilisateur.

```
tags:
  # le nom du tag
  localization:
    # la traduction du tag
    fr: Localisation
    en: Localization
```

1.4 Les référentiels (*references*)

references : Un ensemble d'informations permettant de préciser le contexte de la mesure ou de l'observation.

En déportant ces informations dans des fichiers **references**, on évite la répétition d'informations. On utilisera la clef d'une information pour y faire référence.

- **internationalizationName** : (cf.internationalisationName)
- **internationalizedColumns** : permet de relier des colonnes contenant une information et ses traductions
- **internationalizationDisplay** : permet de définir pour chaque langue comment le référentiel s'affiche dans l'interface.
 - **pattern**: L'expression qui génère la chaîne d'affichage.

```
pattern:
  en: "the name: {colonneMonNom}"
  fr: "le nom: {colonneMonNom}"
  ...:
```

- **keyColumns** : Un tableau des noms des colonnes faisant partie de la clef primaire.
- **tags** : Un tableau des noms des étiquettes (déclaré dans la partie cf. Les étiquettes)souhaité sur la référence pour filtrer l'affichage de la liste des références
- **columns** : (cf. Les colonnes du fichier)
- **computedColumns** : (cf. Les colonnes calculées)
- **dynamicColumns** : (cf. Les colonnes dynamiques)

1.4.1 Les colonnes du fichier

columns : Chaque *columns* est une colonne se trouvant dans le fichier du référentiel. Par conséquent, elle doit avoir le même nom que dans le fichier. - **checker** permet de valider une colonne ou préciser son format (cf checker)

- **defaultValue** : la valeur par défaut si aucune valeur n'est fournie. (cf. groovy)
- **presenceConstraint** : (cf.presenceConstraint)
- **tags** : Un tableau des noms des étiquettes (déclaré dans la partie cf. Les étiquettes)souhaité sur la colonne pour filtré l'affichage du tableau

1.4.2 Les colonnes calculées

computedColumns : Une *computedColumns* est une colonne qui n'est pas présente dans le fichier et dont la valeur est une constante ou le résultat d'un calcul. - **tags** : Un tableau des noms des étiquettes (déclaré dans la partie cf. Les étiquettes)souhaité sur la colonne calculée pour filtrer l'affichage du tableau - **checker** permet de valider une colonne ou préciser son format (cf checker) - **computation**: section de calcul de la donnée - **presenceConstraint** : (cf.presenceConstraint)~~

1.4.3 Les colonnes dynamiques

dynamicColumns : Une *dynamicColumns* est un ensemble de colonnes dont le nom est la concaténation d'un préfixe et d'une valeur dans un référentiel.

Par exemple s'il existe un référentiel "propriétés" avec les valeurs (couleur, catégorie, obligatoire), on pourrait avoir dans un autre référentiel (en utilisant le préfixe "pts_") pts_couleur, pts_catégorie et pts_obligatoire, en les déclarant comme *dynamicColumns*.

- **tags** : Un tableau des noms des étiquettes (déclaré dans la partie cf. Les étiquettes)souhaité sur la colonne dynamique pour filtrer l'affichage du tableau
- **headerPrefix** : on définit la chaîne de caractères servant à identifier les colonnes dynamique se trouvant dans le fichier
- **internationalizationName** : (cf.internationalisationName)
- **presenceConstraint** : (cf.presenceConstraint)
- **reference**: nom de la référence listant les colonnes dynamiques attendues
- **referenceColumnToLookForHeader** : nom de la colonne listant les noms de colonnes dynamiques attendues

1.5 Définition de référentiels hiérarchiques

compositeReferences : Une référence composite est créée en indiquant un lien parent-enfant entre un ou plusieurs référentiels ou une récursion sur un référentiel. Cela permet de générer une clef hiérarchique qui sera utilisée pour afficher hiérarchiquement ces référentiels (par exemple pour les autorizations ou pour le dépôt sur un repository).

- **internationalizationName** : (cf.internationalisationName)
 - **internationalizationName** : (cf.internationalisationName)
 - **reference**: Le nom du référentiel de jointure
 - **parentRecursiveKey** : nom de la colonne parent se trouvant dans le même référentiel que la colonne enfant (dans le cas d'un référentiel qui fait référence à lui-même).
 - **parentKeyColumn** : nom de la colonne parent se trouvant dans le référentiel enfant.

1.6 Les types de données (*dataTypes*)

dataTypes :

- **internationalizationName** : (cf.internationalisationName)
- **tags** : Un tableau des noms des étiquettes (déclaré dans la partie cf. Les étiquettes)souhaité sur le types de donnée pour filtrer l'affichage de la liste des dataTypes.
- **data** : description de l'organisation de l'enregistrement des données dans la base de données sous la forme de composantes de variables. On utilise comme identificateur le nom de la variable. (cf. data)
- **format** : description du format du fichier (cf. format)
- **uniqueness** : C'est là qu'on définit une contrainte d'unicité, en listant la liste des *variable components* qui composent la clef. Si un fichier possède des lignes en doublon avec lui-même il sera rejeté.
- **validations**: Cette section permet de rajouter des validations sur les données. On donnera un nom à chacune des validations en l'utilisant comme clef.
 - **checker**: (cf. checker)
 - **internationalizationName** (cf. internationalizationName)

- **authorization** : Cette section permet de définir les informations sur lesquelles on posera les droits. Elle est aussi utilisée pour le dépôt sur un repository (cf. authorization)
- **repository** : Permet la gestion du dépôt des fichiers par période et contexte en se basant sur les informations de la section (authorization)

1.6.1 data

Les données du fichier sont enregistrées dans des composants et sont liées soit à des informations des colonnes, soit à des informations de l'en-tête. On peut aussi générer des données calculées (computedComponents)

- **tags** : Un tableau des noms des étiquettes (déclaré dans la partie cf. Les étiquettes)souhaité sur le data pour filtrer l'affichage de le dataType.
- **components** : On utilise comme clef le nom de la composante
 - **tags** : Un tableau des noms des étiquettes (déclaré dans la partie cf. Les étiquettes)souhaité sur le components pour filtrer l'affichage de le dataType.
- **computedComponents** : On utilise comme clef le nom de la composante calculée
 - **tags** : Un tableau des noms des étiquettes (déclaré dans la partie cf. Les étiquettes)souhaité sur le computedComponents pour filtrer l'affichage de le tableau du dataType.
- **chartDescription**: permet de définir les données utilisées pour générer des graphes
 - **value**: nom de la composante portant la valeur
 - **unit**: nom de la composante portant l'unité
 - **standardDeviation**: nom de la composante portant l'écart type
 - **gap**: durée à partir de laquelle on considère qu'il y a un saut de données
 - **aggregation**: composante de variable utiliser pour agréger les données
 - * **variable**: nom de la variable
 - * **composante**: nom de la composante

1.6.1.1 Components

- **checker** (cf. checker)
- **defaultValue** (cf. groovy)

1.6.1.2 ComputedComponents

- **checker** (cf. checker)
- **computation** : (cf. computation) comme clef.
- **checker:** (cf. checker)
- **internationalizationName** : (cf. internationalisationName)

1.6.2 format

- **constants** : définition des informations présentes dans l'en-tête du fichier
 - **rowNumber** : numéro de la ligne
 - **columnNumber** : numéro de la colonne
 - **exportHeader** : Un nom de colonne fictif (pour les expressions groovy)
 - **headerName** : Pour les informations présentes sous la ligne d'en-tête le nom de la colonne (à la place du numéro de la colonne) ; par exemple pour une ligne d'unité ou de contraintes.
 - **boundTo**: permet de relier l'information à une composante de variable.
- **headerLine** : numéro de la ligne des noms des colonnes
- **firstRowLine** : numéro de la première ligne de données du tableau
- **columns** : Liste des colonnes du fichier CSV de données
 - **header** : “nom de la colonne se trouvant dans le fichier”
 - **boundTo** : (cf. boundTo).
- **repeteadColumns** : Définit des colonnes dont le nom répond à un pattern donné contenant des informations.
 - **boundTo** : (cf. boundTo).
 - **exportHeader** : Un nom de colonne fictif (pour les expressions groovy)
 - **headerPattern** : une expression régulière dont les groupes sont liés à des composantes de variables dans la section tokens
 - * **tokens**: un tableau de token dans l'ordre de leur groupe dans l'expression régulière. La valeur du groupe sera utilisée comme valeur de la colonne fictive.
 - **boundTo** : (cf. boundTo).
 - **exportHeader** : Un nom de colonne fictif (pour les expressions groovy)

1.6.3 Authorization

- **dataGroups** : Un regroupement des variables (projection) pour leur donner des droits différents. Une ligne du fichier est découpée en autant de lignes qu'il y a de *dataGroups*.
 - **data** : Liste des noms des **variables** définis dans **data**
 - **label** : nom du groupe de données
- **authorizationScopes** : On définit des composantes de contexte. Elles doivent être liées à des référentiels composites (checker Reference) ce qui permettra de les sélectionner dans un arbre pour limiter la portée de l'autorisation. On donnera un nom à chaque 'authorizationScopes en l'utilisant comme clef.
 - **variable** : nom d'un *data*
 - **component** : nom d'un *component* dans un *data*
 - **___internationalizationName**: (cf.internationalisationName)
- **timeScope** : On définit des composantes de temporelles (checker Date) pour limiter les autorisations à des intervalles de dates. Si les valeurs correspondent à une agrégation temporelle, il faudra préciser la durée de la période de aggregation dans le champ duration du checker (par défaut 1 jour).
- **columnsDescription** : décrit comment dans le panneau des autorisations, chacun des rôles (depot, suppression publication..), doit être affiché.
 - **display**: visibilité de la
 - **title**: le nom du rôle
 - **internationalizationName**: (cf.internationalisationName)
 - **withDatagroups** : true si le rôle peut ne porter que sur certaines variables
 - **withPeriods** : true si le rôle peut être limité à certaines périodes. Par défaut, la configuration suivante sera appliquée pour afficher les autorisations. Vous devrez la redéfinir entièrement pour lui apporter des changements :

```
---
admin:
  display: true
  title: admin
  withPeriods: false
  withDataGroups: false
  internationalizationName:
    en: Administration
    fr: Administration
```

```

delete:
  display: true
  title: delete
  withPeriods: false
  withDataGroups: false
  internationalizationName:
    en: Deletion
    fr: Suppression
depot:
  display: true
  title: depot
  withPeriods: false
  withDataGroups: false
  internationalizationName:
    en: Deposit
    fr: Dépôt
extraction:
  display: true
  title: extraction
  withPeriods: true
  withDataGroups: true
  internationalizationName:
    en: Extraction
    fr: Extraction
publication:
  display: true
  title: publication
  withPeriods: false
  withDataGroups: false
  internationalizationName:
    en: Publication
    fr: Publication

```

1.6.4 Repository

- **filePattern** : Permet de définir une expression régulière pour remplir les champs du formulaire automatiquement. Chaque “groupe de l’expression correspond à un **token** ou un **authorizationScope**, qu’on ordonne

- **authorizationScope** : Permet de définir quel groupe correspond à quel **authorizationScopes** (la clef est le nom de l'**authorizationScopes**)
- **startDate** : Section de la date de début
 - `__token` : numéro du groupe ()
- **endDate** : Section de la date de fin
 - `__token` : numéro du groupe ()

1.7 Sections génériques

1.7.1 InternationalisationName

Pour internationaliser les noms et clefs, on utilise la section **internationalizationName**

```
internationalizationName:
  en: expression en anglais
  fr: expression en français
  ...:
```

1.7.2 Groovy

La section groovy accepte trois paramètres

- **expression** : une expression groovy (pour le checker GroovyExpression doit renvoyer true si la valeur est valide)
- **references** : une liste de référentiels pour lesquels on veut disposer des valeurs dans l'expression
- **datatypes** : une liste de datatypes pour lesquels on veut disposer des valeurs dans l'expression

La différence entre une section groovy de la section params d'un checker **groovy** et une section groovy de la section transformation de la section params, tient dans le fait que pour un checker groovy l'expression renvoyée est un booléen tandis que dans la transformation l'expression groovy renvoie une nouvelle valeur.

Les sections **defaultValue** sont des expressions groovy ; par exemple : - 42 - “quarante-deux” - true - 9.8*datum.masse - datum.date.day+” “+datum.date.time”

1.7.3 Checker

checker: c’est la section où on réalise les validations des données et leur typage.

- **name:** il y a plusieurs vérifications possibles :
 - vérifier la nature type d’un champ (float, integer, date) (Integer, Float, Date)
 - vérifier que la valeur vérifie une expression régulière (RegularExpression)
 - ajouter un lien avec un référentiel (Reference)
 - vérifier un script
 - * le script renvoie true (groovy.expression)
 - * le script renvoie une valeur (defaultValue, computation.expression, transformation.groovy)
- **params:** section permettant de passer des paramètres au validateur

1.7.3.1 Contenu de la section params :

- **multiplicity :** La colonne est une valeur (**ONE**), ou un ensemble de valeurs séparées par une virgule (**MANY** ex: 42,36 chat,chien)
- **duration:** pour un checker **Date**, utilisé comme information temporelle (**timeScope**), on peut préciser la durée de la mesure. > Une durée est définie au sens SQL d’un [interval](#) (‘1 HOUR’, ‘2 WEEKS’, ‘30 MINUTES’).
- **groovy:** Pour un checker **GroovyExpression**. (cf. groovy) pour vérifier la valeur avec un script groovy.
- **pattern:** Pour un checker Date ou **RegularExpression** : soit une expression régulière, soit une expression de format de date.
- **reftype:** Pour un checker **Reference** : le nom d’un référentiel
- **required:** true si la valeur doit être fournie
- **transformation**

On peut rajouter une section **transformation** pour modifier la valeur avant sa vérification :

Cette **transformation** peut être configurée avec - **codify** : la valeur sera alors échappée pour être transformée en clé naturelle (Ciel orangé -> ciel_orange) - **groovy** : permet de déclarer une transformation de la valeur avec une expression Groovy (qui doit retourner une chaîne de caractère) (cf. groovy)

- **validations:** cette section permet de rajouter des validations sur plusieurs colonnes en même temps. On donnera un nom à chacune des validations en l'utilisant comme clef de la validation.
 - **internationalizationName :** (cf.internationalisationName)
 - **checker:** (cf. checker)
 - **columns:** un tableau de colonnes sur lesquelles porte la validation.

1.7.3.2 Computation

- **computation:** section permettant la modification de la données avant sa vérification
 - **groovy** (cf. groovy)
 - **references:** un tableau des référentiels à ajouter au contexte de calcul
 - **datatypes:** un tableau des types de données à ajouter au contexte de calcul

1.7.3.3 Presence constraint

- **presenceConstraint :**Pour dire si la colonne doit être présente dans le fichier (**MANDATORY**), ou si on peut l'omettre (**OPTIONAL**)